

Autonomous Mobile Robots (SS26)

Navigation, Localisation and Exploration on the Robile Platform

Project Report

Mohammad Moeed Ahsan

Anurag Tiwari

Haider Qaizar Hussain

September 2026

1. Introduction

The goal of this project was to take things we had only done in simulation during the semester and get them running on the real Robile. The project had three parts that build on each other. First, we ported our potential field planner to the robot and combined it with an A* global planner, so the robot can reach goals anywhere on a map instead of only nearby ones. Second, we wrote our own Monte Carlo localisation (a particle filter) and used it instead of the AMCL node from Nav2. Third, we added frontier-based exploration on top of SLAM, so the robot can map an unknown room by itself.

All of our code is in one ROS 2 package, `potential_field_planner`, with four nodes (the A* planner, the potential field controller, the particle filter and the frontier explorer) and a small helper module for laser geometry. A single launch file starts whichever combination is needed. Figure 1 shows how the pieces connect.

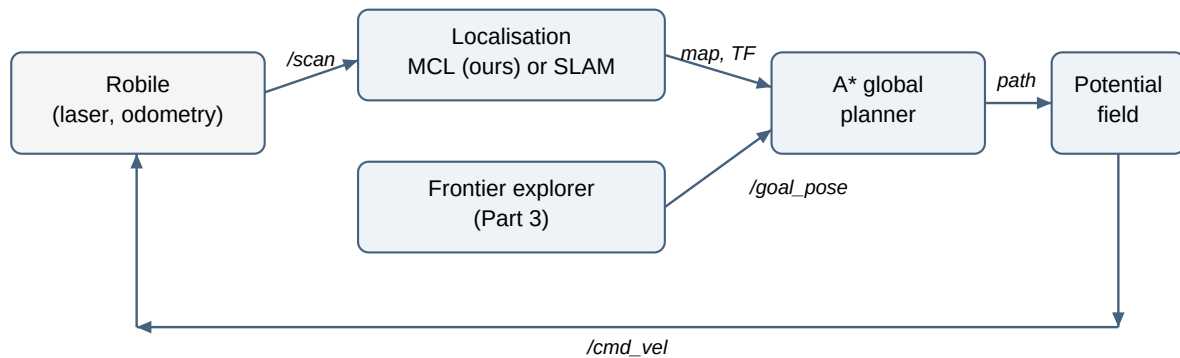


Figure 1: How the nodes fit together. The localisation block is either our particle filter (Parts 1 and 2) or `slam_toolbox` (Part 3).

Something we did not expect going in: a good part of the time went into problems that had nothing to do with the algorithms themselves, such as clock differences between laptop and robot, the mounting of the laser, and odometry slipping when the robot turns quickly. We describe these in Section 6, since they explain several design decisions in the code.

2. Setup

The robot runs its own drivers (`robile_bringup`) and publishes the laser scan, odometry and TF. Everything we wrote runs on a laptop connected over the Robile5G network, using ROS 2 Humble with FastDDS. Our robot (Robile 3) uses ROS domain 3. The footprint is about 0.70 m by 0.47 m, so the half-diagonal, which matters when turning on the spot, is about 0.42 m.

For development we used the Gazebo simulation from the course and, later, a small kinematic simulator of our own. The second one was useful because we could make it deliberately imperfect: a laser mounted upside down, a narrower field of view, odometry that over-reports rotation by 5 to 10 percent, and obstacles that are not in the map. Most of the bugs described in this report were found by running those harder cases before going to the lab.

3. Part 1: Path and Motion Planning

3.1 A* global planner

The planner works on the occupancy grid (5 cm cells). Before searching, every cell gets its distance to the nearest obstacle using a distance transform. A cell is only allowed if it is further than the planning radius (0.30 m) from an obstacle, which is a bit more than half the robot width, so normal doors of about 0.7 m and wider are still passable. On top of that, cells that are close to walls get an extra cost that falls off over 0.35 m. This keeps paths in the middle of corridors instead of hugging the walls, which made a big difference on the real robot.

The search itself is standard A* on an 8-connected grid with the Euclidean distance as heuristic. We do not allow diagonal moves that cut a corner between two blocked cells. The raw path has one point every 5 cm, which is far too dense for the potential field, so we thin it to waypoints roughly 0.4 m apart, checking that the straight line between consecutive waypoints is still free. If the goal lies inside an obstacle or too close to one, the planner moves it to the nearest safe cell instead of failing.

Two things were added after testing. When the robot gets stuck, the planner is asked to replan, and at that point it adds whatever the laser currently sees but the map does not contain (a box, a person). Laser points closer than 0.3 m to a known wall are ignored here, because on the real robot those are just the same wall seen with a small localisation error. Treating them as new obstacles slowly closed off the corridors in one of our early runs.

3.2 Potential field controller

The controller follows the waypoints one after another. The attractive force points at the current waypoint and has unit length, so the gains do not depend on how far away the goal is. The repulsive force is computed from the laser points within 0.8 m of the robot outline (not its centre, since the robot is rectangular) using the usual $(1/d - 1/d_0) / d^2$ shape, summed over the scan and scaled by the angular resolution so it does not depend on how many beams the laser has.

The plain version of this had the problems you would expect. Near walls the repulsion was about fifty times stronger than the attraction, and the robot just sat there turning left and right. We fixed this in three steps. The repulsion is capped at 80 percent of the attraction, so it can steer the robot but never cancel the goal. We add a tangential component (the repulsion from obstacles in front, rotated by 90 degrees towards the goal side), which lets the robot slide around an obstacle instead of stopping in front of it. And when the target is almost straight behind the robot, it keeps turning in the same direction instead of flipping between plus and minus 180 degrees every cycle.

For the last half metre before the goal the Robile drives holonomically: it slides straight onto the goal and turns to the requested heading at the same time. Steering like a car at that distance made the robot wiggle and stop 10 to 15 cm short, which was one of the first things we noticed on the real robot.

There is also a safety layer that checks every velocity command before it is sent. It simulates the next half second of motion, including rotation, and checks whether any laser point would end up within 4 cm of the robot outline. If so, the unsafe part of the command (usually the rotation) is dropped. The controller also stops if the laser or the localisation stops updating, and it reports its state (following, blocked, reached, failed) on a topic so the other nodes and we can see what it is doing.

3.3 Results

In simulation on the lab map, with the map deliberately 2 percent larger than the real world and the laser mounted upside down, the robot reached three goals spread over the whole room with an error of 1 to 13 cm and a heading error of about 0.1 rad, and it went around an unmapped box in the corridor without touching it. On the real robot it followed A* paths across the room and reached the goals set in RViz once localisation and the clock problem (Section 6) were sorted out.

Test (simulation)	Result
10.7 m route across the room	reached, 2 cm from goal
Goal behind an unmapped 0.8 m box	reached, 4 cm from goal, 0.5 m clearance to box
Three goals with required headings	all reached, 1-13 cm, heading error about 0.1 rad
Collisions over all runs	none

4. Part 2: Monte Carlo Localisation

We implemented the basic particle filter from the lecture. The filter itself is a small class that only uses numpy, which let us test it offline on the lab map before wrapping it in a ROS node. The node is a drop-in replacement for AMCL: it reads the map, the laser and the odometry transform, and publishes the map to odom transform, the estimated pose and the particle cloud, so RViz and the planner work with it unchanged.

4.1 Algorithm

Initialisation. 500 particles are drawn from a Gaussian around the pose given with 2D Pose Estimate in RViz. For global localisation (no initial guess) 3000 particles are spread uniformly over the free space of the map.

Motion update. Whenever odometry reports more than 5 cm of travel or 0.05 rad of rotation, the motion since the last update is expressed in the robot frame and applied to every particle with added Gaussian noise. The noise grows with the distance travelled and the angle turned, with four alpha parameters (all 0.2), similar to the odometry model from the lecture. Because the Robile is omnidirectional we move the particles sideways as well, not just forward.

Measurement update. We use a likelihood field. When the map arrives, we precompute for every cell the distance to the nearest wall and turn it into a probability with a Gaussian (sigma 0.2 m) plus a small uniform part for random readings. For each particle, 60 evenly spaced laser beams are projected into the map and their probabilities are combined the same way AMCL does it, as the sum of the cubes. This is less sensitive to a few bad beams than a plain product.

Resampling. Low-variance (systematic) resampling, done only when the effective number of particles drops below half. During global localisation the filter keeps the larger particle set until the particles have actually agreed on one place (spread below 0.5 m). Before we added this, it sometimes collapsed onto the mirror image of the true pose, because the room is almost symmetric.

Estimate. The pose is the weighted mean of the particles, with a circular mean for the heading.

4.2 Results

Offline on the lab map, starting 0.42 m and 0.2 rad off, the filter converged to about 8 cm error within a few metres of driving, and global localisation found the robot to about 10 cm. In the full simulation with navigation, the localisation error stayed below 12 cm on all runs, including the ones with the distorted map. On the real robot, after clicking an approximate pose in RViz, the particle cloud contracts onto the robot within a second or two of moving, the laser scan lines up with the walls, and navigation works exactly as it did with AMCL.

One real-robot issue is worth mentioning here. Our first version only used the yaw of the laser mounting. The Robile laser turned out to be mounted in a way that flips the scan, so our filter saw a mirrored room while AMCL, which uses the full 3D transform, did not. The symptom was a particle cloud that never settled and a robot that did not follow the path. We now project every scan through the full transform (a shared helper used by all nodes).

5. Part 3: Environment Exploration

For mapping we use `slam_toolbox` in asynchronous mode. Our part is the frontier explorer, which decides where the robot should go next, and it sends that goal to the same A* and potential field stack as in Part 1. So the whole navigation chain is reused, only the goals come from the explorer instead of from us.

5.1 Frontier detection and goal selection

A frontier cell is a known free cell next to an unknown one. On every cycle (once per second) the explorer finds all frontier cells in the current SLAM map and groups neighbouring ones into frontiers. Frontiers shorter than 0.4 m are ignored, since they are mostly noise behind chair legs.

For each frontier we look for a goal cell that the robot can actually reach. We compute the travel distance from the robot to every cell with a flood fill over cells the robot fits through, using the same 0.30 m radius as the planner. Candidate goals are all reachable cells within 1 m of any part of the frontier that also have 0.45 m clearance to walls, so the robot can turn around there. The best frontier is the one with the lowest cost: travel distance minus two times the frontier length. Long frontiers promise more new area, so they are worth driving a bit further for. The goal heading points at the frontier, so the robot arrives looking into the unknown part.

A few rules keep the explorer from getting stuck. Places the robot has already visited are not chosen again, frontiers where navigation failed are skipped (and retried once at the end, since the map may have changed), a goal is dropped if its frontier gets explored on the way, and there is a 90 second timeout per goal. The robot does one slow 360 degree turn at the start and another one before it declares the exploration finished. When no reachable frontier is left, the explorer publishes COMPLETE and saves the map.

Our first version required goals to keep 0.45 m clearance from unknown cells as well as from walls. That sounds safe, but next to a frontier every cell is near unknown space, so in a fresh, narrow map the explorer found nothing and stopped after 15 seconds. Only real obstacles push goals and paths away now. Unknown cells cannot be driven through, but they do not count as walls.

5.2 Results

We tested exploration in simulation with an empty map at the start, a 180 degree laser, and odometry that over-reports rotation by 5 percent. In a two-room world connected by a 1 m door the robot mapped 99.8 percent of the floor with 98 percent of the mapped walls within 15 cm of the real ones, and it also got through a narrower 0.85 m door. On the lab layout it covered the whole room in about three and a half minutes. There were no collisions in any of these runs.

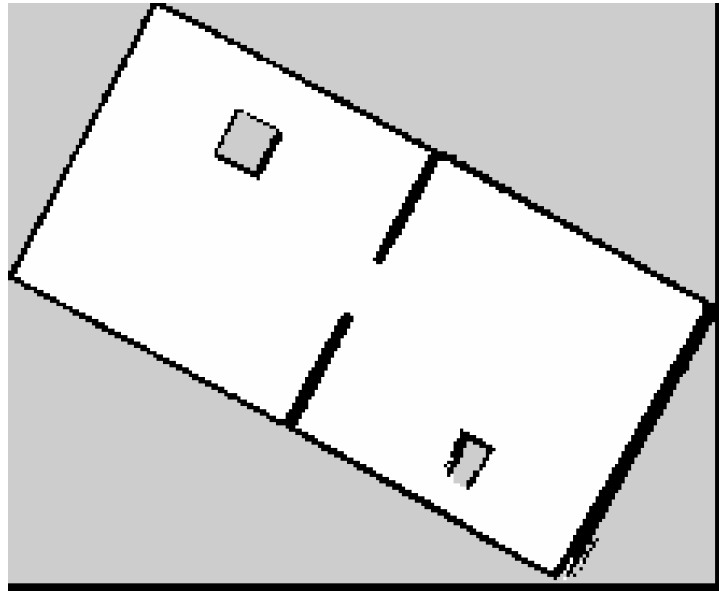


Figure 2: Map built by exploration in the two-room simulation (1 m door, 180 degree laser, 5 percent odometry error).

The first real-robot run went wrong in an instructive way. The robot moved around sensibly, but the saved map contained the room twice, rotated by about 30 degrees against each other. The robot had been turning on the spot at 0.6 to 0.8 rad/s. At that speed the wheel odometry slips, and with 0.2 rad between scans, slam_toolbox matched some scans at the wrong angle. When we built the manual map with teleop we turned slowly, which is why that map was fine. We slowed the look-around turns to 0.3 rad/s and the controller to at most 0.5 rad/s, and made slam_toolbox add a scan every 0.1 rad and 0.1 m. In simulation with 10 percent odometry error the map then stayed consistent. The next run on the real robot finished with a clean map (Figure 3).

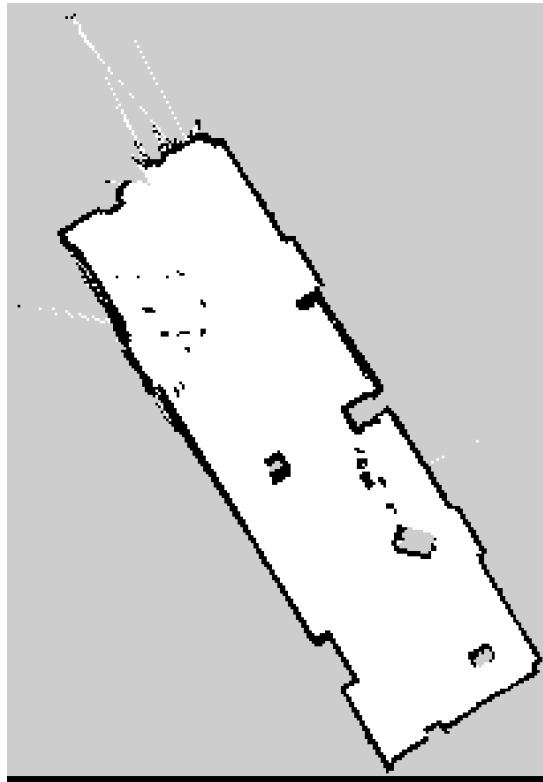


Figure 3: Map of the lab built autonomously by the Robile with our frontier explorer. It is rotated compared to Figure 4 only because the robot started facing a different direction.

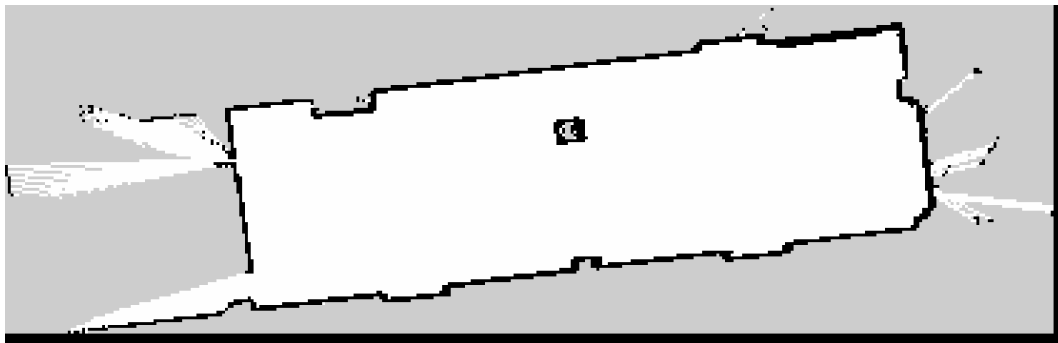


Figure 4: The same room mapped earlier by driving the robot with teleop (the map we used for Parts 1 and 2).

Comparing the two maps, the room shape, the recesses along the walls and the pillar all match. The explored map actually shows more detail, because the robot drove closer to the furniture than we did by hand. The far end is a bit fuzzy where there are chairs and tables, and a few frontiers behind furniture were left unexplored because the robot does not fit there, which is the intended behaviour.

6. Problems on the Real Robot

This section collects the problems that only showed up on the real hardware, since most of them cost us more time than the algorithms.

Robot clock. The robot clock was first about 13 months and later about 9 seconds behind the laptop. ROS compares timestamps across machines, so AMCL threw away scans and our controller refused to move because the pose always looked out of date. Setting the time by hand did not help much because the delay of typing the SSH password ended up in the offset. We ended up running chrony on the laptop as a time server and pointing the robot's timesyncd at it, which fixed it permanently. We also

changed the controller to check whether the pose is still being updated rather than comparing clocks, so a small offset no longer stops the robot.

Laser mounting. As described in Section 4, the laser frame is flipped. Every piece of code that uses the scan now goes through the full 3D transform.

Turning speed and SLAM. Fast in-place rotations corrupted the SLAM map (Section 5.2).

Finding the robot on the map. Setting the initial pose by hand in RViz was harder than expected, especially in a long, almost symmetric room. The global localisation service helps here: spread the particles over the map, turn the robot for 20 seconds, and the cloud usually collapses onto the right place.

Tooling. RViz crashed when started from the VS Code terminal, because VS Code is installed as a snap and passes its own library paths to child processes. We clear those variables in `.bashrc` when running inside VS Code.

7. How to Run

On the robot (over SSH, inside tmux):

```
ros2 launch roble_bringup robot.launch.py
```

On the laptop, after `export ROS_DOMAIN_ID=3` and sourcing the workspace:

```
# Parts 1 and 2: navigation with our particle filter on the saved map
ros2 launch potential_field_planner real_robot_nav.launch.py

# same, but with Nav2 AMCL for comparison
ros2 launch potential_field_planner real_robot_nav.launch.py localization:=amcl

# Part 3: autonomous exploration with SLAM (map saved to ~/explored_map)
ros2 launch potential_field_planner real_robot_nav.launch.py slam:=true
explore:=true
```

RViz opens with the right displays. For navigation, set the pose with 2D Pose Estimate and the goal with 2D Goal Pose. All parameters are in `config/planner_params.yaml`.

8. Conclusion and Possible Improvements

All three parts run on the Robile: the robot navigates to goals on a map using A* and our potential field, it localises itself with our own particle filter, and it can explore and map a room on its own. The main lesson for us was that the simulator hides a lot. Timestamps, sensor mounting and wheel slip all behaved differently on the real robot, and building a simulator that reproduces those problems on purpose was the most useful thing we did for testing.

With more time we would add the adaptive part of AMCL (changing the number of particles with KLD sampling and injecting random particles when the filter gets lost), consider the expected information gain of a frontier instead of only its length, and look at using our particle filter together with the explored map directly after exploration, so the robot can switch from mapping to navigation without restarting.